Symbiosis Institute of Technology, Nagpur

Bachelor of Technology
in
Computer Science and Engineering

Detecting Fraudulent Ethereum Accounts from On-Chain Transaction Behaviour Using Machine Learning and the KDD Process

Submitted By

Rugved Gadge
23070521051

Semester: VII
Section: A

Course

Predictive Analytics

Guide

Deepak Asudani

October 2026

Symbiosis International (Deemed University), Pune

ABSTRACT

Ethereum is a public, programmable blockchain whose openly readable transaction history makes it attractive both for legitimate use and for fraud, such as phishing, scams and money laundering. Manually identifying suspicious accounts does not scale, so this project investigates whether supervised machine learning can classify an Ethereum account as fraudulent or legitimate from aggregated, account-level transaction behaviour. The work follows the Knowledge Discovery in Databases (KDD) process and is implemented in Python using pandas, NumPy, scikit-learn, imbalanced-learn, XGBoost, SHAP, Matplotlib and Seaborn in a Jupyter Notebook.

The notebook starts from a dataset of 9,841 accounts and 51 columns. After removing 25 duplicate-address records, three identifier columns and seven constant columns, 9,816 accounts and 41 columns remain, of which 22.2% are labelled fraudulent. Categorical token-type features are one-hot encoded, 37 skewed features receive a signed-logarithm transformation, 10 highly correlated features are removed and 25 features are capped at training-set percentiles, giving 50 model inputs. All transformation parameters are learned from the 7,852-account training set only. Logistic Regression, Decision Tree, Random Forest and XGBoost were compared with five-fold stratified cross-validation against a majority-class baseline; Random Forest and XGBoost were then tuned by randomised search on average precision, and a decision threshold of 0.60 was chosen from out-of-fold training predictions.

On the untouched test set of 1,964 accounts, the tuned XGBoost model obtained an accuracy of 0.9924, precision of 0.9930, recall of 0.9725, F1-score of 0.9826, ROC-AUC of 0.9993 and PR-AUC of 0.9980, with a confusion matrix of [[1525, 3], [12, 424]]. SHAP analysis ranked ERC-20 activity, particularly the total number of ERC-20 transactions, as the most influential group of features. The project also provides a validated new-account prediction pipeline and dashboard. On a 1,200-account batch, it predicted 80 accounts as fraud and 1,120 as legitimate, with a mean predicted fraud probability of 21.72%.

Diagnostics in the notebook show that about 5.5% of cross-validation validation rows have an identical feature pattern in the training folds and that a few repeated patterns are entirely fraudulent, so the near-perfect scores are interpreted with caution. The results come from a single offline split of one labelled dataset and do not establish production performance; external, time-based validation is identified as essential future work.

Keywords: Ethereum, fraud detection, blockchain analytics, XGBoost, SHAP, KDD, class imbalance, explainable machine learning.

TABLE OF CONTENTS

1. INTRODUCTION AND LITERATURE REVIEW 5
1.1 Background and Problem Statement 5
1.2 Objectives and Scope 5
1.3 Related Work 5
1.4 Research Gap 6
2. SYSTEM ANALYSIS AND ARCHITECTURE 6
2.1 Existing and Proposed Approaches 6
2.2 Requirements 6
2.3 System Architecture and Data Flow 6
3. DATASET AND DATA UNDERSTANDING 7
3.1 Source, Size and Target 7
3.2 Class Balance, Quality and Diagnostics 7
4. METHODOLOGY AND IMPLEMENTATION (KDD) 8
4.1 Selection and Preprocessing 9
4.2 Transformation 9
4.3 Data Mining: Models, Imbalance, Tuning and Threshold 9
4.4 Evaluation and Interpretation 10
5. ALGORITHMS AND EVALUATION SETUP 10
5.1 XGBoost and Threshold Decision 10
5.2 Experimental Setup and Metrics 10
6. RESULTS AND DISCUSSION 11
6.1 Model Comparison, Tuning and Threshold 11
6.2 Final Test Performance 12
6.3 SHAP Interpretation 13
7. NEW-ACCOUNT PREDICTION AND TESTING 14
8. LIMITATIONS, FUTURE SCOPE AND CONCLUSION 15
8.1 Limitations 15
8.2 Future Scope (not implemented) 16
8.3 Conclusion 16
REFERENCES 16
APPENDIX: INSTALLATION AND ROC / PR CODE 17

LIST OF FIGURES

Figure 2.1: System architecture from account data to classification 7
Figure 3.1: Target class distribution (0 = legitimate, 1 = fraud) 8
Figure 4.1: KDD process flow as applied in this project 8
Figure 4.2: Preprocessing and transformation pipeline with counts 9
Figure 6.1: XGBoost out-of-fold threshold sweep on training data 11
Figure 6.2: Cross-validated precision, recall, F1 and PR-AUC of the candidate models 12
Figure 6.3: Confusion matrix on the test set (threshold 0.60) 12
Figure 6.4: SHAP summary plot (1,000 test accounts) 13
Figure 6.5: Mean absolute SHAP values of the top features 14
Figure 7.1: Prediction dashboard for a 1,200-account batch 15

LIST OF TABLES

Table 1.1: Summary of related research 5
Table 2.1: Requirements, environment and assumptions 6
Table 3.1: Dataset dimensions through the pipeline 7
Table 3.2: Leakage and redundancy diagnostics 8
Table 4.1: Selected hyper-parameters 10
Table 6.1: Five-fold CV results on the training set (mean) 11
Table 6.2: Test-set results (1,964 accounts, 436 fraud) 12
Table 6.3: Top five features by mean |SHAP| (log-odds units) 13
Table 7.1: Testing status (only performed checks are claimed) 15

LIST OF ABBREVIATIONS

API – Application Programming Interface; AUC – Area Under the Curve; CV – Cross-Validation; EDA – Exploratory Data Analysis; ERC-20 – Ethereum fungible-token standard; FN/FP/TN/TP – false/true negative/positive; KDD – Knowledge Discovery in Databases; OOF – Out-of-Fold; PR-AUC – area under the precision-recall curve (average precision); RF – Random Forest; ROC – Receiver Operating Characteristic; SHAP – SHapley Additive exPlanations; SMOTE – Synthetic Minority Over-sampling Technique; XGBoost – eXtreme Gradient Boosting.

1. INTRODUCTION AND LITERATURE REVIEW

1.1 Background and Problem Statement

Ethereum is a public, programmable blockchain whose openly readable transactions support both legitimate use and fraud such as phishing, Ponzi schemes and money laundering [1][2]. Accounts are pseudonymous, and flagged-address lists are reactive and incomplete. Supervised machine learning can learn from labelled accounts which aggregated behavioural features (transaction counts, values, activity span, ERC-20 token activity) separate fraudulent from legitimate accounts.

Problem statement. Given account-level Ethereum features with a binary label FLAG (0 = legitimate, 1 = fraudulent), build and evaluate a classifier that estimates fraud probability, applies a justified threshold, explains its predictions and scores new account records with the same preprocessing used in training.

1.2 Objectives and Scope

Objectives: clean and transform the data following the KDD process [3] with parameters learned from training data only; compare Logistic Regression, Decision Tree, Random Forest and XGBoost; tune the leaders and choose a threshold without the test set; evaluate once on held-out data; interpret with SHAP; and provide a validated new-account prediction pipeline.

Scope: offline, account-level classification on one labelled dataset; no graph modelling, live data or deployment.

1.3 Related Work

Table 1.1: Summary of related research

Study	Method	Findings (as reported)
Farrugia et al. [2], 2020	XGBoost on account features; 2,179 flagged + 2,502 normal accounts	10-fold CV: accuracy 0.963, AUC 0.994; top features: activity span, Ether balance, minimum value received
Ostapowicz and Żbikowski [4], 2019	RF, SVM, XGBoost on >300,000 accounts; sensitivity analysis	Precision and recall reported as suitable for an anti-fraud rule
Chen et al. [5], 2018	Account and opcode features for smart-contract Ponzi schemes	High accuracy; detection possible at contract creation
Wu et al. [6], 2019	Transaction-network embedding (trans2vec) for phishing	Embedding effective for phishing identification; needs graph construction
Weber et al. [7], 2019	Bitcoin AML: LR, RF, MLP, GCN on >200K transactions	Random Forest superior among tested methods

Tree ensembles suit tabular transaction data: Random Forests average decorrelated trees [8], while gradient boosting adds trees sequentially to correct errors [9], as implemented by XGBoost [10]. Imbalance is handled by SMOTE [11] or class weights.

1.4 Research Gap

Existing account-level studies show tree models work, but this project adds (i) explicit leakage and duplicate diagnostics, (ii) train-only selection of preprocessing, model and threshold, (iii) cautious SHAP interpretation, and (iv) a schema-validated new-account pipeline. It does not claim a new algorithm.

2. SYSTEM ANALYSIS AND ARCHITECTURE

2.1 Existing and Proposed Approaches

Current practice relies on community labels, hand-written rules, graph-embedding methods [6] and contract-level analysis [5]; these are reactive, brittle, data-heavy or limited to one fraud type. The proposed system is a supervised account-level pipeline: training-fitted preprocessing, a tuned XGBoost model, a 0.60 decision threshold, SHAP explanation and a batch dashboard. It is a research prototype in a Jupyter Notebook.

2.2 Requirements

Table 2.1: Requirements, environment and assumptions

Category	Details
Functional	Load and inspect data; clean duplicates, identifiers and constant columns; stratified split; train-only transformation; CV model comparison and tuning; threshold selection; one-time test evaluation; SHAP; new-account input validation , probability and label output, dashboard
Non-functional	Reproducibility (seed 42); leakage control; interpretability; efficiency (5-fold CV took 0.6 s for XGBoost, 4.0 s for Random Forest); robustness through schema validation; portability via requirements.txt
Software	Python not recorded Jupyter / VS Code, pandas [12], NumPy [13], scikit-learn [14], imbalanced-learn [15], XGBoost [10], SHAP [16], Matplotlib, Seaborn (requirements.txt is unpinned)
Hardware	not recorded the data (<10,000 rows) and models are small, so a standard laptop suffices
Assumptions	Labels are correct; new-account features are computed exactly as in training; static snapshot; single random split

2.3 System Architecture and Data Flow

Figure 2.1 shows the flow: cleaning, split, training-fitted transformation, cross-validated model comparison, tuning, threshold selection, a single test evaluation and SHAP; the same fitted objects then drive the new-account pipeline and dashboard.

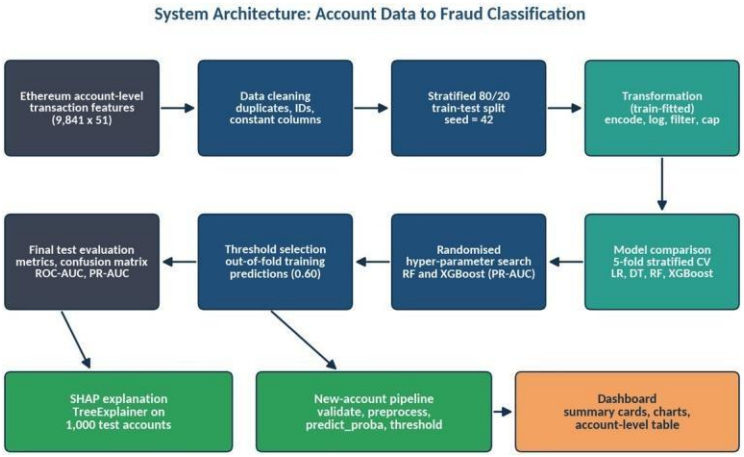


Figure 2.1: System architecture from account data to classification

3. DATASET AND DATA UNDERSTANDING

3.1 Source, Size and Target

matching the illicit count of Farrugia et al. [2] [17], but 7,662 legitimate accounts rather than their 2,502, so it is not identical to that collection and its exact origin must be confirmed. FLAG is 0 = legitimate, 1 = fraudulent (README); such labels mean reported or flagged, not proven fraud.

Table 3.1: Dataset dimensions through the pipeline

Stage	Size	Notes
Raw data	9,841 × 51	3 identifiers + FLAG + 47 candidate predictors
After removing 25 duplicate addresses, 3 identifiers, 7 constant columns	9,816 × 41	FLAG + 40 predictors (38 numeric, 2 text); 2,179 fraud (22.2%)
Training / test (stratified 80/20, seed 42)	7,852 / 1,964	1,743 / 436 fraud
After transformation	50 features	Same columns in both sets

Predictors cover activity timing, sent/received counts, unique counterparties, Ether sent/received values, balance, contract activity, ERC-20 counts, values and token diversity, and two categorical token-type columns (exact definitions to be checked against dataset documentation

3.2 Class Balance, Quality and Diagnostics

Fraud is 22.2% of accounts, so a majority-class baseline scores 0.778 accuracy with zero recall (Figure 3.1). The 25 duplicate addresses had no conflicting labels. After cleaning, 18 columns have missing

values (27.6% of accounts), concentrated in ERC-20 columns: 16 numeric columns are each missing for exactly 829 accounts, suggesting structural missingness (plausibly no ERC-20 activity, to be confirmed). Descriptive medians show fraud-flagged accounts have shorter activity spans (7,545 vs 116,626 minutes), shorter intervals between transactions and smaller values.

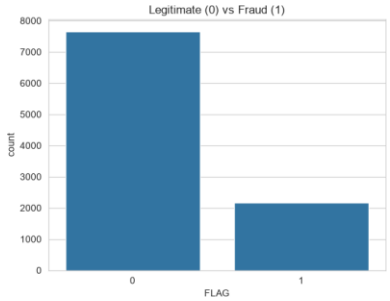


Figure 3.1: Target class distribution (0 = legitimate, 1 = fraud)

Table 3.2: Leakage and redundancy diagnostics

Diagnostic	Observation
Rows identical in all features and label	546 (5.55%); 523 fraud, 23 legitimate
Repeated feature patterns in training	9 patterns covering 433 accounts, none with mixed labels; the largest (216, 102, 96) are all fraud
CV overlap (identical vector in training folds)	5.51% of validation rows on average
Strongest single feature (univariate AUC)	total ether received, 0.815; no direct proxy for the label

No single feature gives the label away, but repeated, uniformly fraudulent patterns mean a random split can put near-copies in both training and test sets. Only address duplicates were removed, so headline scores must be read with this in mind.

4. METHODOLOGY AND IMPLEMENTATION (KDD)

The notebook follows the five KDD phases [3] in Python with pandas, NumPy, scikit-learn, imbalanced-learn, XGBoost and SHAP (Figures 4.1 and 4.2).

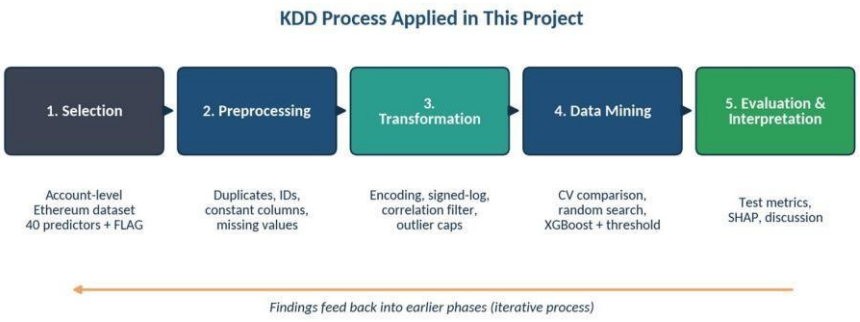


Figure 4.1: KDD process flow as applied in this project

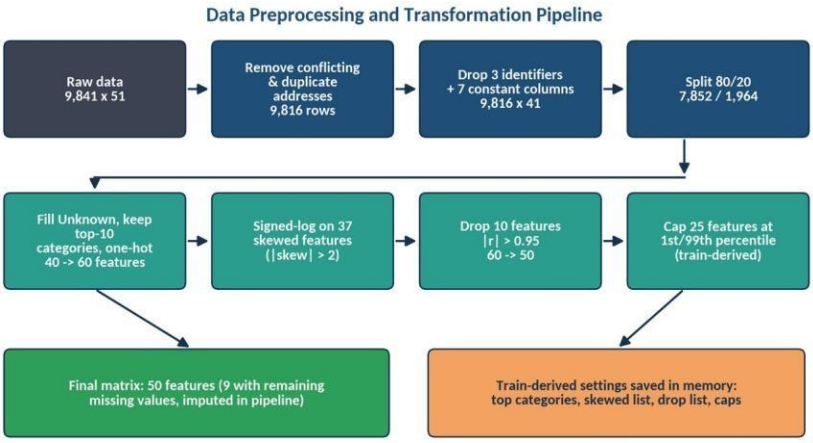


Figure 4.2: Preprocessing and transformation pipeline with counts

4.1 Selection and Preprocessing

FLAG is the target; Unnamed: 0, Index and Address are identifiers and are dropped, as are seven constant ERC-20 columns. One row per address is kept. Missing numeric values are intentionally left for median imputation inside the modelling pipelines so statistics come from training folds only.

4.2 Transformation

After the stratified split, all parameters are learned from the training set: (i) the two token-type columns get Unknown for missing values, top-10 categories plus Other, then one-hot encoding (40 → 60 features); (ii) 37 of 38 continuous features with |skewness| > 2 get a signed logarithm (average | skew| 46.66 → 10.46); (iii) 10 features with |r| > 0.95 are dropped (60 → 50); (iv) 25 features are clipped to training 1st/99th percentiles.

$$x' = sign(x) \cdot ln(1 + |x|) \tag{4.1}$$

$$x'' = min(\, max(\, x',\, q_{0.01}\,),\, q_{0.99}\,) \tag{4.2}$$

Techniques **not** used: removal of identical feature rows, PCA, global scaling (a scaler appears only in the Logistic Regression pipeline), SMOTE in the final XGBoost model, calibration, and time- or group-based validation.

4.3 Data Mining: Models, Imbalance, Tuning and Threshold

A majority-class baseline and four models were compared by 5-fold stratified cross-validation on the training set, with median imputation (and scaling for Logistic Regression) inside each pipeline. SMOTE [11] was applied inside folds for Logistic Regression, Decision Tree and Random Forest; XGBoost uses scale_pos_weight = 6,109 / 1,743 ≈ 3.50. Random Forest and XGBoost were tuned by randomised search [18] (20 candidates × 5 folds, scored on average precision). Out-of-fold probabilities on the training set were used to sweep thresholds 0.10–0.90 and pick the best F1 (ties → higher recall): 0.60 for XGBoost.

Table 4.1: Selected hyper-parameters

Model	Settings
XGBoost (selected)	n_estimators 233, max_depth 8, learning_rate 0.0649, min_child_weight 1, subsample 0.825, colsample_bytree 0.908, gamma 1.76, reg_alpha 0.0067, reg_lambda 0.274, scale_pos_weight 3.50, seed 42
Random Forest (tuned)	208 trees, max_depth 25, max_features sqrt, min_samples_split 8, min_samples_leaf 1

4.4 Evaluation and Interpretation

The chosen pipeline is refitted on all training data and evaluated once on the test set. SHAP TreeExplainer [19] is run on a 1,000-account test sample, with mean absolute SHAP ranking and a direction check.

5. ALGORITHMS AND EVALUATION SETUP

5.1 XGBoost and Threshold Decision

XGBoost [10] models the score as a sum of K regression trees added sequentially with learning rate η, minimising a regularised logistic-loss objective:

$$F(x) = \Sigma_{k=1..K}\, f_k(x), \qquad Obj = \Sigma_i\, l(\, y_i,\, F(x_i)\,) + \Sigma_k\, \Omega(f_k) \tag{5.1}$$

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda\, \Sigma\, w_j^2 + \alpha\, \Sigma\, |w_j|, \qquad p = 1 / (1 + e^{-F(x)}), \qquad \hat{y} = 1[\, p \geq 0.60\,] \tag{5.2}$$

A second-order expansion gives optimal leaf weights $w_j^* = -G_j / (H_j + \lambda)$ and a split gain penalised by γ. Trees need no feature scaling, capture non-linearities and interactions, handle missing values and support exact TreeSHAP attributions [19]; this makes XGBoost appropriate for skewed tabular transaction data, without implying it is universally superior (the Random Forest was equally competitive here). SHAP assigns each feature a Shapley-value contribution [16] with $F(x) = \varphi_0 + \Sigma\varphi_i$, on the log-odds scale for XGBoost; it shows model association, not causation.

5.2 Experimental Setup and Metrics

Seed 42 is used for the split, folds, SMOTE, search, models and SHAP sampling; library versions beyond pandas 3.0.6 and hardware are not recorded [not recorded] Encoding, skew, correlation and cap parameters were learned on the full training set before cross-validation, which can make CV estimates slightly optimistic; the test set played no role in any decision.

$$Precision = TP/(TP+FP),\ Recall = TP/(TP+FN),\ F1 = 2PR/(P+R),\ Accuracy = (TP+TN)/N \tag{5.3}$$

In fraud screening a false negative is missed fraud and a false positive is a wrongly flagged account, so precision and recall are reported alongside ROC-AUC [20] and PR-AUC (average precision), which is more informative under imbalance [21]; accuracy alone is misleading.

6. RESULTS AND DISCUSSION

6.1 Model Comparison, Tuning and Threshold

Table 6.1: Five-fold CV results on the training set (mean)

Model	Accuracy	Precision	Recall	F1	PR-AUC
Baseline (majority)	0.778	0.000	0.000	0.000	0.222
Logistic Regression	0.989	0.966	0.983	0.974	0.995
Decision Tree	0.988	0.970	0.974	0.972	0.951
Random Forest	0.993	0.996	0.972	0.984	0.999
XGBoost	0.995	0.995	0.983	0.989	0.998

All models beat the baseline; Random Forest and XGBoost lead. After tuning, CV PR-AUC was 0.9987 (XGBoost) versus 0.9986 (Random Forest), a gap far below the fold standard deviation, so the models are indistinguishable on this evidence; XGBoost was chosen for its higher precision at its threshold and faster training. Figure 6.1 shows the XGBoost threshold sweep and Figure 6.2 compares the models: precision rises and recall falls as the threshold increases, and F1 is nearly flat between 0.45 and 0.80, so 0.60 (precision 0.9936, recall 0.9799, F1 0.9867 out-of-fold) is not a sharp optimum.

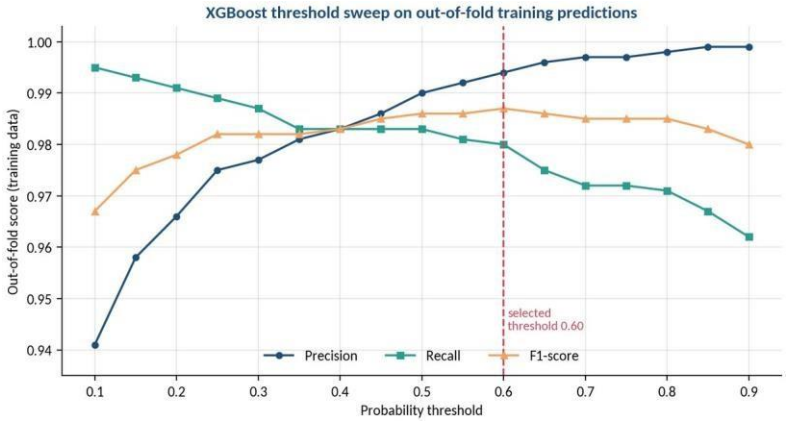


Figure 6.1: XGBoost out-of-fold threshold sweep on training data

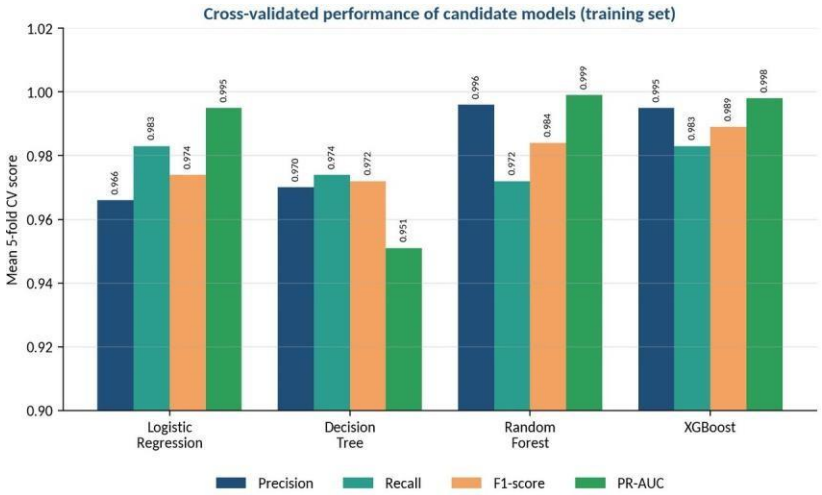


Figure 6.2: Cross-validated precision, recall, F1 and PR-AUC of the candidate models

6.2 Final Test Performance

Table 6.2: Test-set results (1,964 accounts, 436 fraud)

Metric	Value
Accuracy	0.9924
Precision	0.9930
Recall	0.9725
F1-score	0.9826
ROC-AUC	0.9993
PR-AUC	0.9980
Threshold	0.60
Confusion matrix [[TN, FP], [FN, TP]]	[[1525, 3], [12, 424]]

The model detected 424 of 436 fraud accounts and wrongly flagged 3 of 1,528 legitimate accounts (false-positive rate 0.2%, miss rate 2.75%). Test scores are close to the out-of-fold estimates (F1 0.9826 vs 0.9867). The ROC and precision-recall curves are not plotted in the notebook, so they are **[Pending – figures not available; code in Appendix]**.

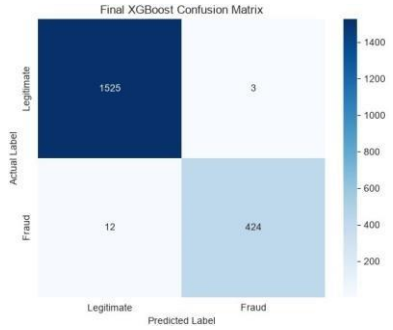


Figure 6.3: Confusion matrix on the test set (threshold 0.60)

6.3 SHAP Interpretation

Table 6.3: Top five features by mean |SHAP| (log-odds units)

Rank	Feature	Mean |SHAP|	Reading (association only)
1	Total_ERC20_tnxs	2.76	Higher counts push toward fraud (corr +0.56)
2	ERC20_most_sent_token_type*	1.59	Likely blank-category indicator; presence pushes toward fraud
3	Time_Diff_between_first_and_last	1.06	Longer activity span pushes toward legitimate
4	ERC20_total_ether_sent	0.68	Higher values push toward fraud
5	ERC20_most_rec_token_type_Unknown	0.56	Missing received-token type pushes toward fraud

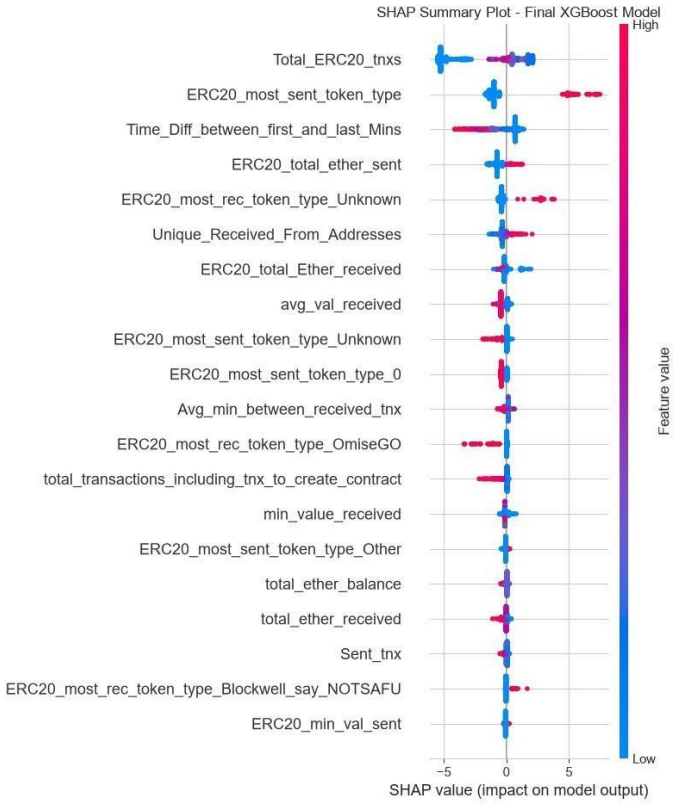


Figure 6.4: SHAP summary plot (1,000 test accounts)

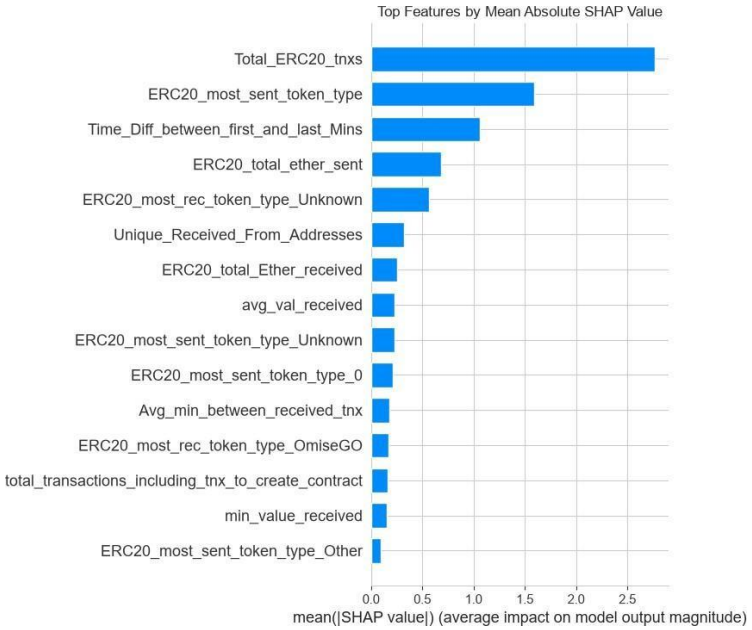


Figure 6.5: Mean absolute SHAP values of the top features

Total_ERC20_tnxs is the most influential feature, followed by token-type indicators and the activity span. *Caution:* the second-ranked name most likely belongs to the one-hot indicator for a blank token-type category whose trailing underscore was stripped by name cleaning several top features therefore reflect whether ERC-20 data exist, which may partly be a dataset artifact. Importance is not causation, and a feature such as many distinct senders may also describe a legitimate exchange account.

Balanced reading. Performance is very high on a random split, but about 5.5% of validation rows have identical copies in training and nine uniformly fraudulent patterns cover 433 training accounts, so part of the score reflects repeated patterns; the effect was not quantified. The evidence supports separability within this dataset, not detection of new fraud on the live network.

7. NEW-ACCOUNT PREDICTION AND TESTING

Section 11 of the notebook scores new account records from a batch input. It normalises column names, rejects any input containing FLAG, requires the 40 predictor columns (extras ignored), then replays the stored training settings: Unknown/Other categories and one-hot encoding, signed-log on the stored skewed list, dropping the stored correlated list, clipping to stored limits and aligning to the 50 training columns. The pipeline's predict_proba gives Fraud_Probability; values at or above 0.60 are labelled Fraud, otherwise Legitimate. Median imputation covers missing numbers and unseen tokens become Other. Predictions are valid only if new-account features are computed exactly as in training

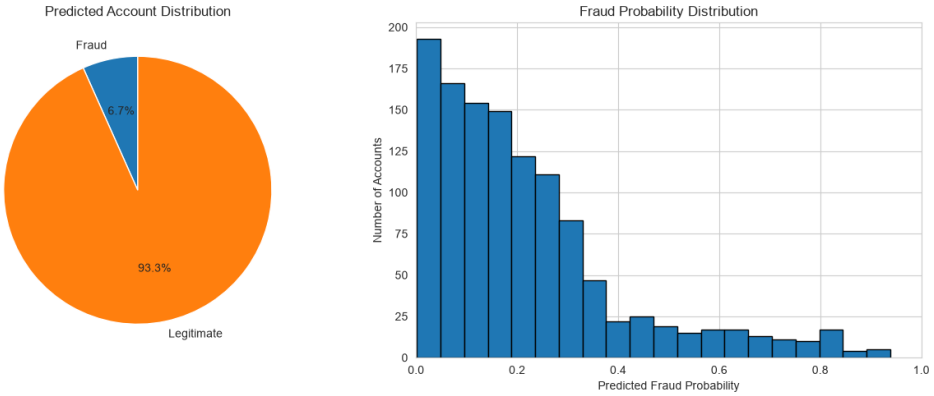


Figure 7.1: Prediction dashboard for a 1,200-account batch

For the 1,200-account batch, the dashboard showed 80 predicted fraud (6.67%), 1,120 predicted legitimate and a mean predicted fraud probability of 21.72%. These values describe model outputs and should not be interpreted as verified fraud outcomes or real-world fraud prevalence. The 1,500-account batch produced 326 predicted fraud (21.73%), 1,174 predicted legitimate and a mean predicted fraud probability of 35.47%.

Table 7.1: Testing status (only performed checks are claimed)

Test	Status
Train/test columns identical; no text, infinite or constant columns; no train–test overlap	Performed (notebook output)
Test probabilities in [0, 1] (0.0002–0.9997); labels and confusion matrix at 0.60	Performed
1,200-account batch passes input validation; predictions and dashboard produced	Performed
FLAGrejected; missing column rejected; 1,500-row file; unseen categories; non-numeric text; empty or single-row file	Pending – not executed
Repeatability across runs; new-account preprocessing reproduces X_train for training rows	Pending – not executed

8. LIMITATIONS, FUTURE SCOPE AND CONCLUSION

8.1 Limitations

- Labels are reported or flagged accounts; dataset source and licence are unrecorded; real-world fraud prevalence is unknown.
- Repeated feature patterns and a single random split (436 fraud test cases, no confidence intervals) may make results optimistic; no external or time-based validation.
- Static snapshot, so changing fraud behaviour is not modelled; probabilities are uncalibrated; the threshold has no cost model.
- SHAP shows association only and several top features are token-presence indicators; account-batch predictions demonstrate pipeline operation, not verified accuracy; testing is limited and no security testing was done.

9.8 Comparative Prediction Results Across Three Datasets

This section compares the model's outputs across the held-out evaluation set and two account-batch prediction runs. Predicted labels use the project's selected decision threshold of 0.60. These are model classifications, not independently verified fraud outcomes.

Table 9.1: Comparative prediction summary

Dataset / run	Accounts	Predicted fraud	Predicted legitimate	Fraud prediction rate	Mean fraud probability
Held-out test set	1,964	427*	1,537*	21.74%*	Not reported
Account batch A	1,200	80	1,120	6.67%	21.72%
Account batch B	1,500	326	1,174	21.73%	35.47%

*For the held-out test set, the confusion matrix is [[1525, 3], [12, 424]], giving 427 predicted fraud and 1,537 predicted legitimate accounts at the selected threshold. The account-batch rows summarize dashboard outputs.

Figure 9.1: Predicted account counts



9.8 Comparative Prediction Results Across Three Datasets (continued)

Figure 9.2: Fraud prediction rate by dataset



Interpretation

The model assigned the highest share of fraud labels to account batch B (21.73%), followed by the held-out test set (21.74%, rounded) and account batch A (6.67%). The mean predicted fraud probability was 21.72% for account batch A and 35.47% for account batch B. A predicted label is determined by whether the estimated probability meets the 0.60 threshold; the mean probability is a separate summary and can differ from the predicted-label share.

These figures describe model outputs only. Predicted labels should not be treated as verified fraud cases, and prediction percentages should not be interpreted as real-world fraud prevalence. The held-out test metrics reported in Section 6 remain the appropriate measures of model performance on labelled data.

8.2 Future Scope (not implemented)

Validate on newer, independently labelled data with time-based and pattern-aware splits and repeated-feature removal; add confidence intervals; compare further models and graph features [6]; calibrate and choose cost-based thresholds; add automated tests; and deploy as a secure, monitored API.

8.3 Conclusion

Following the KDD process, the project cleaned 9,841 accounts to 9,816, built a 50-feature training-fitted representation, compared four classifiers and selected a tuned XGBoost model with a train-derived 0.60 threshold. On the held-out test set it reached accuracy 0.9924, precision 0.9930, recall 0.9725, F1 0.9826, ROC-AUC 0.9993 and PR-AUC 0.9980 (3 false positives, 12 false negatives), and SHAP highlighted ERC-20 activity, notably Total_ERC20_tnxs. The schema-validated prediction pipeline and dashboard produced batch predictions successfully. The main contribution is a transparent, leakage-conscious workflow; its conclusions are bounded by one offline split of one labelled dataset, label uncertainty and repeated patterns, so independent, time-based validation is the necessary next step before any real-world use.

REFERENCES

[1] V. Buterin, “A next-generation smart contract and decentralized application platform (Ethereum white paper),” 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[2] S. Farrugia, J. Ellul, and G. Azzopardi, “Detection of illicit accounts over the Ethereum blockchain,” Expert Syst. Appl., vol. 150, Art. no. 113318, 2020, doi: 10.1016/j.eswa.2020.113318.

[3] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, “From data mining to knowledge discovery in databases,” AI Mag., vol. 17, no. 3, pp. 37–54, 1996.

[4] M. Ostapowicz and K. Żbikowski, “Detecting fraudulent accounts on blockchain: A supervised approach,” arXiv:1908.07886, 2019.

[5] W. Chen, Z. Zheng, J. Cui, E. C. H. Ngai, P. Zheng, and Y. Zhou, “Detecting Ponzi schemes on Ethereum: Towards healthier blockchain technology,” in Proc. World Wide Web Conf. (WWW), Lyon, France, 2018, pp. 1409–1418, doi: 10.1145/3178876.3186046.

[6] J. Wu et al., “Who are the phishers? Phishing scam detection on Ethereum via network embedding,” arXiv:1911.09259, 2019 (published in IEEE Trans. Syst., Man, Cybern., Syst.).

[7] M. Weber et al., “Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics,” in KDD ’19 Workshop on Anomaly Detection in Finance, Anchorage, AK, USA, Aug. 2019, arXiv:1908.02591.

[8] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[12] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf., 2010, pp.

56–61.

[13] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[14] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.